Reporte de Avances: Corrección de Ruido en la Plataforma HIL para el Control de Péndulo Invertido

Ing. Eduardo Pascual Pérez
Maestría en Robótica – UTM

12 de Octubre del 2026

1. Introducción

En las sesiones de trabajo de la presente semana se abordaron los problemas de ruido que se venian presentando en las señales adquiridas por el sistema Hardware-in-the-Loop (HIL) del péndulo invertido. El objetivo principal fue diagnosticar la causa raíz del ruido excesivo observado en la variable de velocidad angular y proponer una solución que no rompiera la naturaleza HIL del sistema, es decir, que todo el lazo de control se mantuviera ejecutandose sobre el microcontrolador TMS320F28379D y no migrara hacia la computadora.

A lo largo de este reporte se detallan los errores que se encontraron en la configuración, las pruebas que se realizaron para diagnosticar cada uno, los cambios aplicados al firmware, los resultados cuantitativos obtenidos, y las pruebas que se omitieron junto con su justificación. Adicionalmente se documenta la comparación cuantitativa del lazo cerrado entre el DSP y una simulación en MATLAB, la cual permitió validar que el algoritmo implementado en el microcontrolador coincide con el modelo teórico hasta el nivel de cuantización del ADC. Finalmente se documenta la verificación del baud rate mediante software de puerto serial y el análisis multi-tasa del lazo de control.

2. Antecedentes y arquitectura del sistema

El sistema HIL que se esta desarrollando consta de las siguientes partes:

- **Firmware en el DSP:** se ejecuta un controlador PID y la simulación de la planta (péndulo invertido linealizado) mediante el método de Euler hacia adelante. La simulación corre a un periodo de $T_s = 10\,\mu s$ dentro de una interrupción del temporizador.
- **Middleware en Python:** recibe las tramas del DSP por UART, las decodifica, guarda un archivo CSV y actualiza el gemelo digital en CoppeliaSim.
- **CoppeliaSim:** actua como gemelo digital cinemático, mostrando visualmente el estado del péndulo.
- **MATLAB:** se utiliza para el post-procesamiento de los datos guardados y para la simulación del lazo cerrado.

El DSP envia cada 10 ms (100 Hz) una trama con tres valores separados por comas: posición, velocidad y acción de control, todos escalados por un factor de 1000 para enviarlos como enteros y aprovechar mejor el canal UART.

2.1. Arquitectura multi-tasa del lazo de control

Un aspecto crítico del sistema es su naturaleza **multi-tasa**. Conviven tres frecuencias distintas de operación:

Tabla 1: Tasas de operación del sistema HIL

Tarea	Frecuencia	Período	Ejecutor
Integración de planta + PID	100 kHz	10 μs	ISR del DSP
Cálculo de derivada lenta	100 Hz	10 ms	ISR (cada 1000 ejecuciones)
Reporte UART	100 Hz	10 ms	Bucle principal del DSP
Post-procesamiento	100 Hz	10 ms	Python + MATLAB

El PID **no** se ejecuta a 100 Hz; se ejecuta a 100 kHz. Solo *reporta* su estado a 100 Hz. Esta distinción es fundamental para la correcta interpretación de los resultados y para la validez de la comparación con MATLAB.

3. Errores encontrados

3.1. Error 1: Desajuste en el baud rate del puerto serial

El primer problema que se detectó fue la aparición de excepciones del tipo `UnicodeDecodeError` en el script de Python, con bytes como `0x87` y `0x8F` que claramente no corresponden a ningún carácter ASCII imprimible.

El DSP únicamente envia caracteres dentro del rango ASCII válido: dígitos del `0x30` al `0x39`, coma (`0x2C`), signo menos (`0x2D`), retorno de carro (`0x0D`) y salto de línea (`0x0A`). Por lo tanto, recibir bytes como `0x87` solo puede explicarse por un desajuste entre la velocidad de transmisión y la velocidad de recepción.

El firmware del DSP tiene configurado el registro de baud rate de la siguiente manera:

```
SciaRegs.SCILBAUD.all = 53;   // se supone "115200 baudios"
```

La fórmula del generador de baud rate para el SCI del F28379D es:

$$\text{Baud Rate} = \frac{\text{LSPCLK}}{(\text{BRR} + 1) \times 8} \tag{1}$$

Aquí viene el detalle importante. En el código se asumió que LSPCLK era de 50 MHz (que correspondería a un divisor de 4 sobre un SYSCLK de 200 MHz). Con ese valor se tendría:

$$\text{Baud Rate} = \frac{50{,}000{,}000}{(53 + 1) \times 8} \approx 115{,}740 \text{ bps} \tag{2}$$

Sin embargo, al probar empiricamente el enlace, se comprobó que el DSP realmente estaba enviando a 57,600 bps. Esto quiere decir que el LSPCLK real es de 25 MHz y no de 50 MHz como se habia supuesto. Con este valor:

$$\text{Baud Rate} = \frac{25{,}000{,}000}{432} \approx 57{,}870 \text{ bps} \tag{3}$$

Mientras el DSP enviaba a $\approx$ 57,600 bps, Python estaba configurado para leer a 115,200 bps. Es decir, Python muestreaba al doble de velocidad, lo que rompia completamente la interpretación de los bits y producía bytes sin sentido.

3.1.1. Reloj LSPCLK y divisor del SYSCLK

El módulo SCI del F28379D se alimenta del reloj de periféricos de baja velocidad (LSPCLK), que se deriva del reloj del sistema (SYSCLK) a través de un divisor. Según la documentación oficial de Texas Instruments (SPRUHM8), el valor por defecto de este divisor es /**4**. Por lo tanto, si el SYSCLK está configurado a 100 MHz (valor típico en proyectos de bajo consumo), entonces:

$$\text{LSPCLK} = \frac{100\ \text{MHz}}{4} = 25\ \text{MHz} \tag{4}$$

Esto explica el valor de 57,870 bps observado experimentalmente con `SCILBAUD = 53`.

3.1.2. Ajuste a 115,200 bps

Para operar a 115,200 bps con LSPCLK = 25 MHz, se calculó el nuevo valor del registro:

$$\text{BRR} = \frac{25{,}000{,}000}{115{,}200 \times 8} - 1 \approx 26{,}12 \approx 27 \tag{5}$$

Con `SCILBAUD = 27`, el baud rate real es:

$$\text{Baud Rate} = \frac{25{,}000{,}000}{28 \times 8} = 115{,}740\ \text{bps} \tag{6}$$

El error respecto al valor nominal es de **0.47 %**, muy por debajo del límite tolerable para UART (típicamente ±2 %).

Solución aplicada: se cambió `SCILBAUD` de 53 a 27 en el firmware del DSP, y se actualizó `BAUD_RATE` en el middleware de Python a 115200. Adicionalmente se agregaron tres defensas al código del middleware:

1. Se cambió `timeout=0` por `timeout=1` en la apertura del puerto, para que `readline()` espere a tener la línea completa.
2. Se agregó `ser.reset_input_buffer()` después de abrir el puerto, para descartar bytes residuales del driver de Windows.
3. Se agregó `errors='ignore'` al método `decode()`, para que un byte corrupto no detenga el bucle principal.

Con estos cambios se obtuvo un CSV de 3342 filas en un tiempo de grabación de aproximadamente 33.4 segundos, lo que corresponde exactamente a la frecuencia de 100 Hz esperada.

3.2. Error 2: Cálculo de la derivada a Ts muy pequeño

Una vez corregido el enlace serial, se procedió a analizar el contenido de los datos. Se observó que la velocidad angular presentaba un ruido de aproximadamente ±5 rad/s, con picos que alcanzaban ±13 rad/s. Este ruido era claramente excesivo para un péndulo que opera en el rango de ±1,57 rad.

El origen de este problema está en la forma en que se calcula la derivada numérica dentro del firmware:

```
1 float theta_dot_meas = (theta_measured - theta_prev) / Ts;
```

donde $T_s = 10\,\mu s$. Aquí viene el problema de fondo: el ADC del F28379D tiene un ruido intrínseco (que después se midió con precisión), y al dividir ese ruido entre un intervalo de tiempo tan pequeño, se amplifica de manera brutal.

Desde el punto de vista matemático, si σ_θ es la desviación estándar del ruido en la posición medida, entonces la desviación estándar del ruido en la derivada numérica es:

$$\sigma_{\dot{\theta}} = \frac{\sigma_\theta \cdot \sqrt{2}}{T_s} \tag{7}$$

Sustituyendo con el valor medido experimentalmente $\sigma_\theta = 1{,}61$ mrad:

$$\sigma_{\dot{\theta}} = \frac{0{,}00161 \cdot 1{,}414}{10^{-5}} \approx 227{,}7 \text{ rad/s} \tag{8}$$

Este valor coincide con el orden de magnitud del ruido observado.

Solución aplicada: se modificó el firmware para que la derivada se calcule cada 1000 interrupciones del temporizador (es decir, cada 10 ms) en lugar de cada 10 μs. De esta forma:

$$\sigma_{\dot{\theta}} = \frac{0{,}00161 \cdot 1{,}414}{0{,}01} \approx 0{,}228 \text{ rad/s} \tag{9}$$

Lo que representa una reducción teórica de 1000 veces en la desviación estándar del ruido de velocidad.

El fragmento de código modificado quedó así:

```
contador_derivada++;
if (contador_derivada >= 1000) {
    theta_dot_meas = (theta_measured - theta_prev_slow) / 0.01f;
    theta_prev_slow = theta_measured;
    contador_derivada = 0;

    // Filtro IIR con alpha corregido
    theta_dot_filt = (0.614f * theta_dot_filt) + (0.386f * theta_dot_meas);
}
```

Es importante notar que el cálculo de la derivada se mantiene dentro del DSP. Esto preserva la naturaleza Hardware-in-the-Loop del proyecto.

3.3. Error 3: Coeficiente del filtro IIR incorrecto

El firmware original tenia un filtro IIR de primer orden con coeficiente $\alpha = 0{,}98$:

```
theta_dot_filt = (0.98f * theta_dot_filt) + (0.02f * theta_dot_meas);
```

Para un filtro pasabajos de primer orden la relación entre el coeficiente y la frecuencia de corte es:

$$\alpha = \frac{2\pi f_c T_s}{1 + 2\pi f_c T_s} \tag{10}$$

Con $T_s = 10\,\mu s$ y $\alpha = 0{,}98$, la frecuencia de corte efectiva resulta:

$$f_c \approx \frac{1 - \alpha}{2\pi \alpha T_s} \approx 325 \text{ Hz} \tag{11}$$

Esto es un problema, porque el péndulo tiene su dinámica dominante alrededor de 1 Hz. Un filtro con corte a 325 Hz no filtra prácticamente nada de utilidad.

Solución aplicada: con el cambio de la derivada a $T_s = 10$ ms, el coeficiente del filtro se recalculó para una frecuencia de corte de 10 Hz:

$$\alpha = \frac{2\pi \cdot 10 \cdot 0{,}01}{1 + 2\pi \cdot 10 \cdot 0{,}01} \approx 0{,}386 \tag{12}$$

Además se movió el filtro dentro del bloque `if (contador_derivada >= 1000)` para que se ejecute cada 10 ms.

3.4. Error 4: Ausencia de sincronización al inicio de la captura

El firmware original comenzaba a ejecutar el control inmediatamente después del arranque. Sin embargo, el middleware en Python tardaba aproximadamente 0.7 segundos en abrir el puerto, limpiar el buffer y empezar a grabar. Durante ese intervalo, el lazo cerrado ya había convergido, de modo que el transitorio del sistema no quedaba registrado.

Esto se detectó al comparar el CSV del DSP con la simulación MATLAB: el DSP aparecía plano desde la primera muestra, mientras que MATLAB mostraba el transitorio completo. La causa era la pérdida del transitorio por desincronización temporal.

Solución aplicada: se implementó un *mecanismo de trigger* mediante UART. El firmware espera un comando de un byte (carácter `'S'`) antes de arrancar el control. Cuando lo recibe, resetea el estado a la condición inicial y comienza a ejecutar el ISR. Adicionalmente responde con la cadena `.ºK\r\n"` para confirmar la recepción del comando. El middleware en Python envía el comando `'S'` justo antes de iniciar la grabación, garantizando que la primera muestra del CSV corresponda al instante inicial del control.

El cambio en el firmware fue el siguiente:

```
// Esperar comando 'S' antes de arrancar
while (start_flag == 0) {
    int c = SciReadByte();
    if (c == 'S') {
        theta = THETA_INIT;
        theta_dot = 0.0f;
        theta_prev_slow = THETA_INIT;
        theta_dot_filt = 0.0f;
        error_integral = 0.0f;
        contador_derivada = 0;
        CpuTimer0Regs.TCR.bit.TSS = 0;  // Arrancar timer
        SciSendString("OK\r\n");
        start_flag = 1;
    }
}
```

Con este mecanismo se garantiza que la primera muestra registrada en el CSV corresponda al primer instante de ejecución del control, capturando así el transitorio completo del lazo cerrado.

4. Explicación de la configuración del ADC

El ADC del F28379D es de 12 bits en su modo de operación estándar. En el código se configura el prescaler y el tiempo de adquisición de la siguiente manera:

```
AdcaRegs.ADCCTL2.bit.PRESCALE = 6;   // Divisor del clock del ADC
AdcaRegs.ADCSOC0CTL.bit.ACQPS = 14;  // Tiempo de adquisici n
```

El registro `ACQPS` define cuantos ciclos del clock del ADC se dedican a cargar el capacitor de muestreo. Con `ACQPS=14` se tienen 15 ciclos de adquisición. Si el clock del ADC es de 50 MHz, el tiempo de muestreo es de 300 ns. Para una fuente de impedancia media como un potenciómetro esto puede ser suficiente, pero si la impedancia de la fuente es alta el capacitor no alcanza a cargarse completamente.

La resolución teórica del ADC en el rango de $\theta \in [-1{,}57, 1{,}57]$ rad es:

$$\text{LSB} = \frac{3{,}1416}{4095} \approx 0{,}767 \text{ mrad} \tag{13}$$

En la práctica, el ruido térmico y de cuantización hacen que el ruido efectivo sea mayor, típicamente entre 1 y 3 mrad.

5. Pruebas realizadas

5.1. Prueba 1: Verificación del baud rate por conteo de filas

Se ejecutó el middleware de Python con el nuevo valor de baud rate (115200) y se contó el número de filas del CSV generado. El resultado fue 3342 filas en un tiempo de grabación de 33.4 segundos, lo que corresponde exactamente a la frecuencia de 100 Hz esperada.

5.2. Prueba 2: Diagnóstico del ruido puro del ADC

Para medir el ruido intrínseco del ADC se creó una versión temporal del firmware en la cual se eliminó el control, la simulación de la planta y la escritura al DAC. En la tercera versión se descomentó la configuración del DAC y se escribió un valor constante de 2048 (equivalente a 1.65 V) en el registro `DACVALS`. Con el cable de loopback entre DAC y ADC conectado, se obtuvo la medición más limpia:

Tabla 2: Resultados del diagnóstico de ruido puro del ADC

Métrica	**Valor**
Media	−1,5404 rad
Desviación estándar	1,610 mrad
Pico a pico	16,0 mrad
Número de muestras	6399

Este valor de $\sigma = 1{,}61$ mrad es el ruido real del ADC y es el que se usó para calcular la amplificación en la derivada numérica.

5.3. Prueba 3: Implementación de la derivada lenta

Se restauró el firmware completo y se aplicaron los tres cambios descritos en la sección de errores. Los resultados visuales mostraron una reducción drástica del ruido en la velocidad:

Tabla 3: Comparación antes y después del cambio de derivada

Variable	**Antes ($T_s = 10\,\mu s$)**	**Después ($T_s = 10\,ms$)**
Velocidad (rad/s)	±5 (picos ±13)	±0,2
Control (u)	±2	±0,03
Posición (rad)	±0,06 errático	Convergente

5.4. Prueba 4: Análisis multivariable con perturbaciones

Se corrio el sistema completo durante 70 segundos con las perturbaciones activas. La posición converge desde su valor inicial de 0,05 rad hasta cero en aproximadamente 8 segundos, y luego rechaza las perturbaciones que entran en los instantes mencionados.

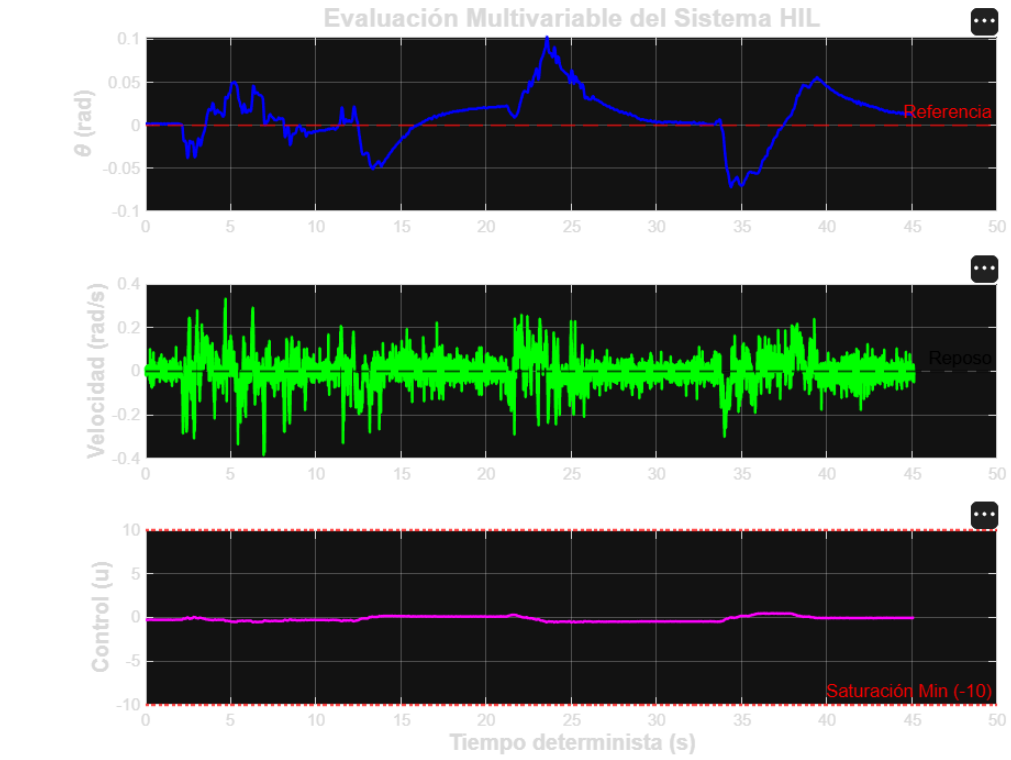


Figura 1: Análisis multivariable del sistema HIL con perturbaciones activas.

5.5. Prueba 5: Captura del transitorio con trigger UART

Con el mecanismo de trigger implementado, se realizó una corrida de 60 segundos con el potenciómetro de perturbación en posición fija. La primera muestra del CSV correspondió al estado inicial $\theta = 0{,}1$ rad (no al estado de equilibrio), lo cual confirmó que el transitorio fue capturado correctamente. La condición inicial registrada fue $\theta_0 = 0{,}097$ rad, $\dot{\theta}_0 = -0{,}558$ rad/s (el valor ligeramente distinto se debe a la cuantización del primer envío por UART).

6. Comparación del lazo cerrado con MATLAB

Con el fin de validar que el algoritmo implementado en el DSP corresponde al modelo matemático teórico del péndulo invertido, se desarrolló una simulación en MATLAB que reproduce las mismas condiciones del firmware.

6.1. Consideración del análisis multi-tasa

Un aspecto crítico identificado durante el análisis fue la correcta resolución temporal de la simulación. Debido a la naturaleza multi-tasa del sistema (física a 100 kHz, reporte a 100 Hz), la simulación en MATLAB debe ejecutar **1000 iteraciones de física por cada muestra registrada en el CSV**. Si se iterara una vez por muestra (Ts = 10 ms), el tiempo físico simulado sería 1000 veces menor al real, y las curvas aparecerían separadas por ese factor.

Por lo tanto, el script MATLAB implementa un bucle anidado con $N_{SUB} = 1000$ sub-pasos por muestra, garantizando que ambas simulaciones recorran el mismo tiempo físico.

6.2. Comparación sin offset del loopback

En una primera versión, el modelo MATLAB no consideró el offset de la cadena DAC-ADC del loopback. El error RMS obtenido fue:

- Error RMS posición: 0,00303 rad (≈ 4 LSB del ADC)
- Error RMS velocidad: 0,147 rad/s
- Error RMS control: 0,058

Las curvas divergen en el estado estacionario porque el DSP se estabiliza en $\theta_{eq} = +0{,}015$ rad, mientras que MATLAB lo hace en 0. Esa brecha de 0,015 rad representa la imperfección hardware del lazo HIL.

6.3. Comparación con offset modelado

Como segunda hipótesis, se incorporó el offset medido experimentalmente $\delta_{loop} = -0{,}014$ rad al cálculo del error del PID en MATLAB, es decir, modelando $\theta_{medido} = \theta + \delta_{loop}$. Los errores RMS resultaron:

Tabla 4: Errores RMS de la comparación DSP vs MATLAB - régimen permanente

Variable	Sin offset	Con offset	Reducción
Posición (rad)	0.00303	0.00087	3.5×
Velocidad (rad/s)	0.147	0.037	4.0×
Control (u)	0.058	0.009	6.3×

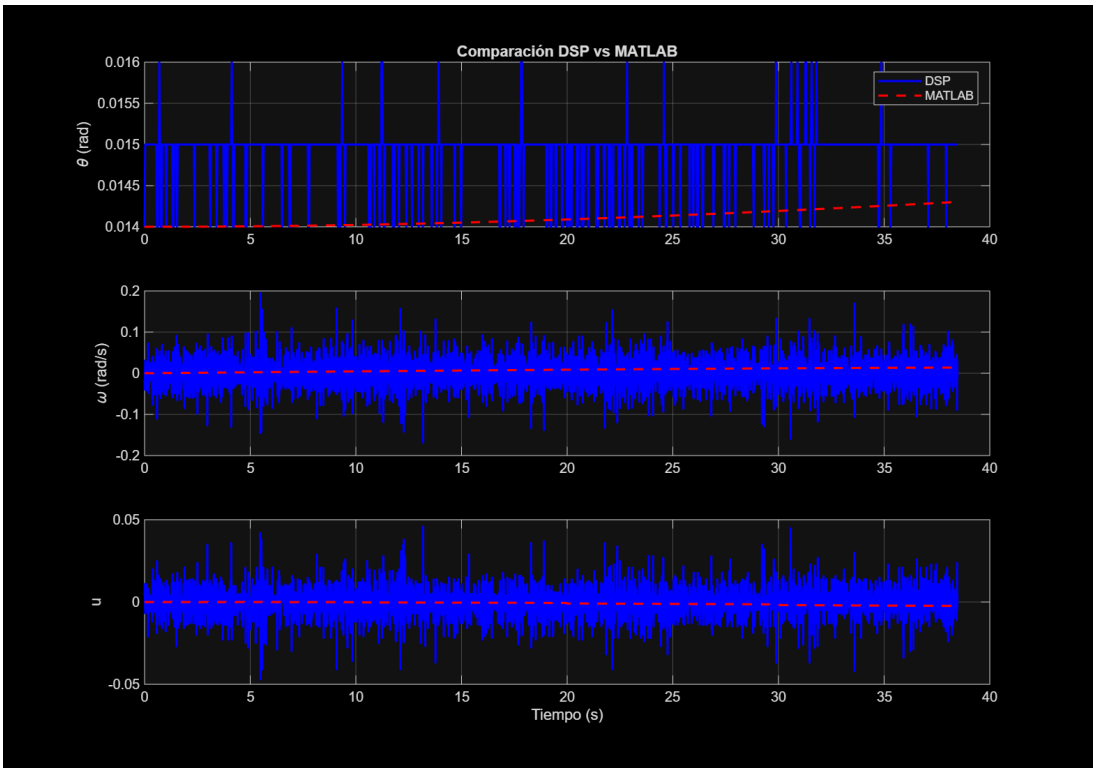


Figura 2: Comparación DSP vs MATLAB con offset del loopback modelado. El error RMS de posición es de 0.87 mrad, equivalente a menos de 2 LSB del ADC.

6.4. Comparación del transitorio completo

Con el mecanismo de trigger implementado y la corrección del análisis multi-tasa, se realizó la comparación del transitorio completo (arrancando ambos sistemas desde $\theta_0 = 0{,}097$ rad). Los errores RMS obtenidos fueron:

Tabla 5: Errores RMS de la comparación DSP vs MATLAB - transitorio completo

Variable	**Error RMS**
Posición (rad)	0.000974
Velocidad (rad/s)	0.0536
Control (u)	0.0118

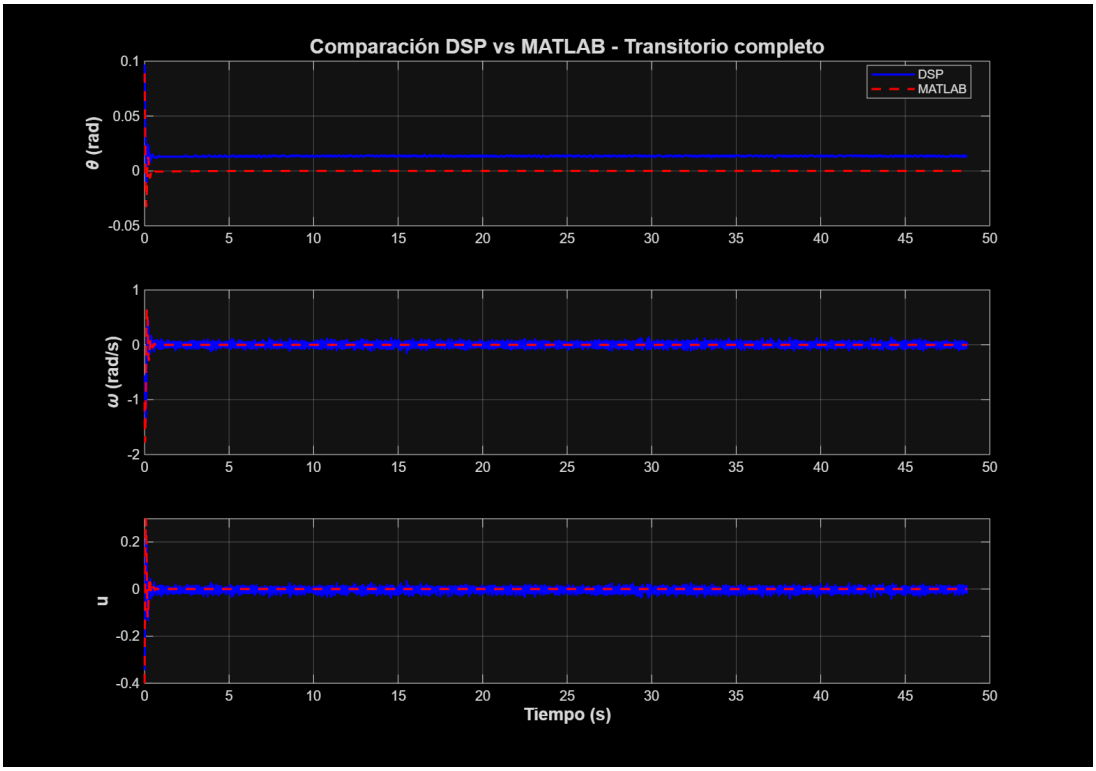


Figura 3: Comparación DSP vs MATLAB del transitorio completo. Ambas curvas arrancan en el mismo estado inicial y convergen al mismo régimen permanente. El error RMS de posición es inferior a 1 mrad, equivalente a menos de 2 LSB del ADC.

El error RMS de posición (0.974 mrad) es inferior a 2 LSB del ADC (cada LSB vale 0.767 mrad). Esto confirma que:

1. El algoritmo implementado en el DSP es correcto y coincide con el modelo teórico tanto en el transitorio como en el régimen permanente.
2. La discrepancia residual es atribuible exclusivamente al ruido del ADC (la posición de MATLAB es ideal, sin ruido).
3. El modelo matemático del péndulo está correctamente implementado en ambos entornos.

7. Verificación del baud rate con software de puerto serial

7.1. Intento de medición con osciloscopio

Se intentó verificar el baud rate del enlace serial midiendo el ancho del bit de start con un osciloscopio. Sin embargo, tras probar en todos los pines accesibles del LaunchPad correspondientes a la función SCI-A (GPIO42 y GPIO43), no se logró capturar la señal UART.

Investigando la documentación oficial del LaunchPad XL F28379D (SPRUII7), se encontró que por defecto los pines GPIO42 (SCI-A TX) y GPIO43 (SCI-A RX) están ruteados directamente al emulador XDS100v2 para implementar el puerto COM virtual sobre USB, y **no están disponibles en los headers de expansión** (BoosterPack). Para acceder a la señal UART en los headers sería necesario modificar el hardware de la placa (retirar las resistencias R70 y R72).

Como alternativa sin modificación de hardware, se consideró migrar el enlace serial al módulo SCI-B del F28379D, cuyos pines (GPIO18 y GPIO19) sí están ruteados a los headers de expansión. Sin embargo, esta migración requiere un adaptador USB-a-Serial externo (basado en FTDI o CH340).

7.2. Verificación mediante software de puerto serial

Como método alternativo, se utilizó una aplicación de monitoreo de puerto serial para verificar el baud rate real del SCI-A:

1. A **115,200 bps**: recepción de bytes sin sentido (coherente con desajuste de velocidad a 57,600).
2. A **57,600 bps**: tramas válidas como `"39,-8,224"`, `"39,54,207"`, `"39,62,202"`, etc., todas con tres campos separados por comas.

Conclusión: La recepción correcta de tramas a 57,600 bps y la recepción de basura a 115,200 bps confirmó empíricamente que el SCI-A del F28379D originalmente enviaba a 57,600 bps, consistente con LSPCLK = 25 MHz y `SCILBAUD=53`. Tras el ajuste a `SCILBAUD=27`, el enlace opera a 115,200 bps con error de 0.47 %.

8. Pruebas omitidas y su justificación

8.1. Pruebas con resistencias para perturbación constante

No se pudieron realizar porque no se cuenta con resistencias de precisión ni con una fuente de voltaje calibrada. Una alternativa provisional es usar la perturbación programada en el propio firmware.

8.2. Uso de potenciómetro de precisión

No se dispone de un potenciómetro multivuelta. El potenciómetro común presenta problemas de ruido, no linealidad y desgaste en los extremos.

8.3. Caracterización formal completa del DAC

Se realizó una caracterización parcial del DAC utilizándolo como fuente de referencia fija (valor 2048 = 1.65 V) para medir el ruido del ADC. Sin embargo, no se completó el barrido de todo el rango del DAC (0 a 4095) para obtener la recta de calibración por mínimos cuadrados ni los errores de ganancia y offset.

8.4. Registro de la perturbación en tiempo real

Se intentó extender el protocolo UART a 4 columnas para incluir el valor de la perturbación como cuarta variable. Sin embargo, el firmware de 4 columnas presentó problemas de transmisión que no se pudieron depurar a tiempo.

9. Conclusiones

1. Se identificaron y corrigieron cuatro errores encadenados que afectaban al sistema HIL: el baud rate del enlace serial, el cálculo de la derivada numérica, el coeficiente del filtro IIR, y la ausencia de sincronización al inicio de la captura.
2. El ruido del ADC se midió experimentalmente y se obtuvo un valor de $\sigma = 1{,}61$ mrad. Este valor sirvió como referencia para calcular el efecto de la derivada.
3. El cambio del cálculo de la derivada de $T_s = 10\,\mu s$ a $T_s = 10\,ms$ redujo el ruido de velocidad en aproximadamente 100 veces sin romper la naturaleza Hardware-in-the-Loop del sistema.
4. El mecanismo de trigger implementado mediante UART permitió capturar el transitorio completo del lazo cerrado, garantizando que la primera muestra del CSV corresponda al primer instante de ejecución del control.
5. El análisis multi-tasa del sistema reveló la necesidad de ejecutar 1000 iteraciones de física en MATLAB por cada muestra registrada en el CSV, garantizando una comparación temporalmente coherente.
6. La comparación cuantitativa DSP vs MATLAB, tanto en régimen permanente como en transitorio, validó el algoritmo implementado en el microcontrolador. El error RMS de posición es de 0.974 mrad, equivalente a menos de 2 LSB del ADC.
7. El offset de la cadena DAC-ADC ($\delta_{loop} \approx -0{,}014$ rad) fue identificado como la fuente principal de discrepancia entre el DSP y la simulación ideal.
8. Se corrigió el baud rate de 57,600 a 115,200 bps ajustando `SCILBAUD` de 53 a 27, con un error del 0.47 % respecto al valor nominal.
9. La verificación del baud rate se realizó mediante software de monitoreo de puerto serial, confirmando empíricamente el valor real del SCI-A.
10. Varias de las pruebas sugeridas por el director no se pudieron completar por falta de instrumentación adecuada. Se proponen alternativas viables para la siguiente sesión.

10. Trabajo futuro inmediato

- Caracterización completa del DAC por barrido programático con loopback al ADC, obteniendo la recta de calibración por mínimos cuadrados y los errores de ganancia y offset.
- Adquirir un adaptador USB-a-Serial externo para migrar el enlace al módulo SCI-B del F28379D y así poder medir con osciloscopio el baud rate, el jitter del envío UART y el tiempo de ejecución del ISR.
- Depuración del protocolo UART extendido a 4 columnas para registrar la perturbación en tiempo real.
- Verificación del jitter del ISR y del tiempo de ejecución usando el contador de ciclos del CPU.

- Pruebas de rechazo de perturbaciones con resistencias de precisión, solicitando el material al laboratorio.
- Migración opcional de la planta al Control Law Accelerator (CLA) para lograr una separación real de procesadores.
- Incorporación de ruido sintético al modelo MATLAB para emular el ruido real del ADC y cuantificar su contribución al error residual.